Probability density estimation
STA510

Rizzo chapters 12.1 and 12.2

Probability density estimation

- *Provided a sample* $x_1, \ldots, x_n$ from some unknown probability density function $f(x)$; *how do we estimate* $f(x)$?
- Many applications:
  - useful for reporting/presenting results from statistical analysis or simulation.
  - checking/validating simulation code.
  - sometimes we need to estimate the value of the density $f(x)$ (or even derivatives, e.g. $f'(x)$) in some point x.
  - many more...

Two main avenues: histogram and KDE

- *histograms* (hist() in R)
- *kernel density estimates* (KDE) (density() (and others) in R)
- example: Given som data in a vector x:

```
length(x)
```

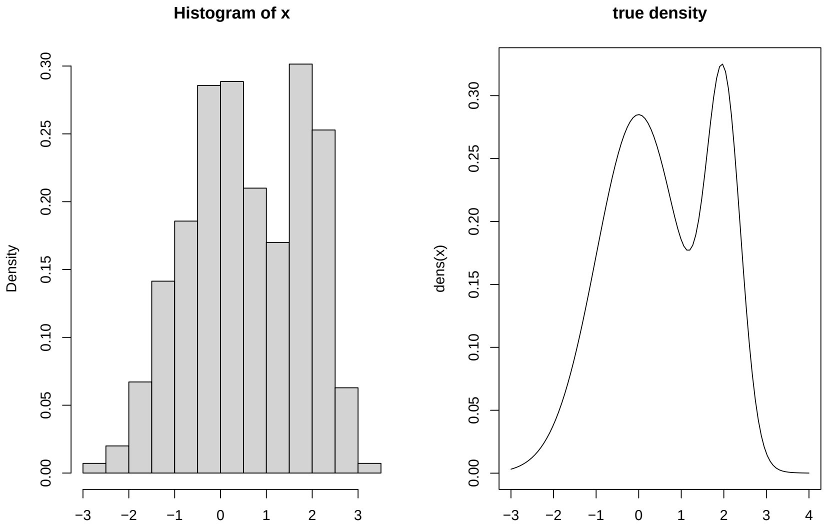
```
## [1] 1400
```

```
print(round(x[1:8],2))
```

```
## [1] -0.56 -0.23  1.56  0.07  0.13  1.72  0.46 -1.27
```

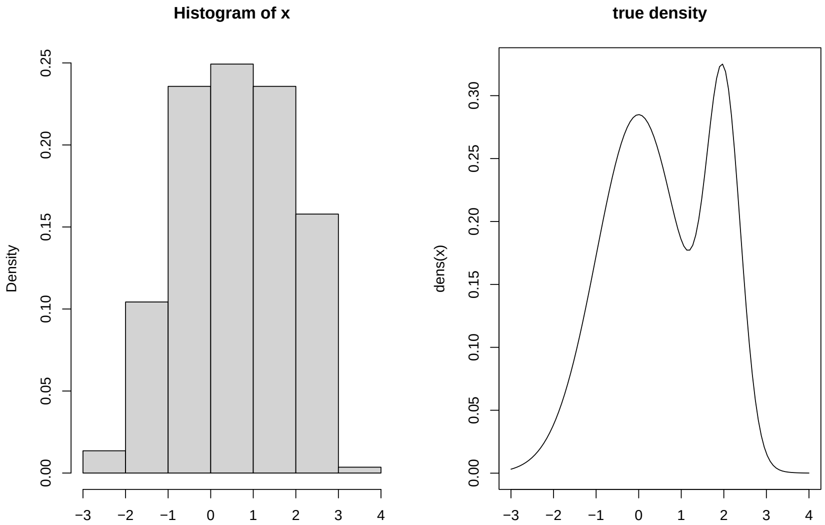
Intro example: Histogram (default)

```r
par(mfrow=c(1,2))
hist(x,probability=TRUE)
curve(dens,from=-3,to=4,main="true density")
```



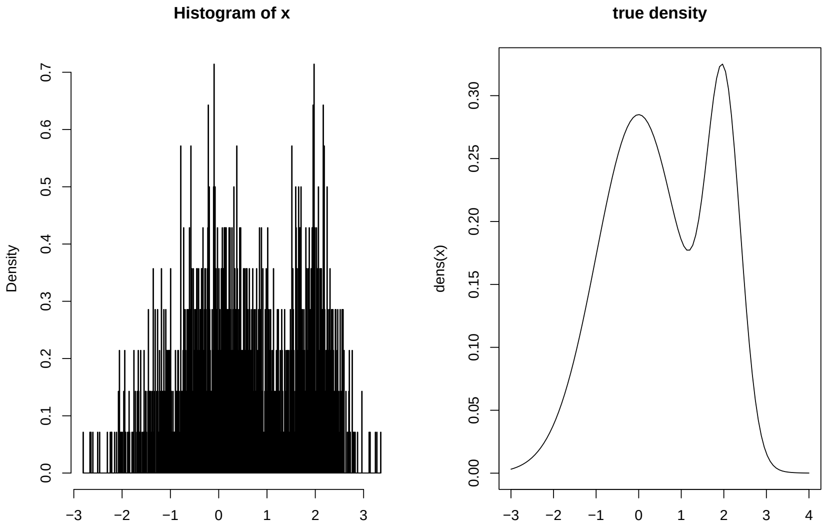
Intro example: Histogram (too few bins)

```
par(mfrow=c(1,2))
hist(x,breaks=5,probability=TRUE)
curve(dens,from=-3,to=4,main="true density")
```



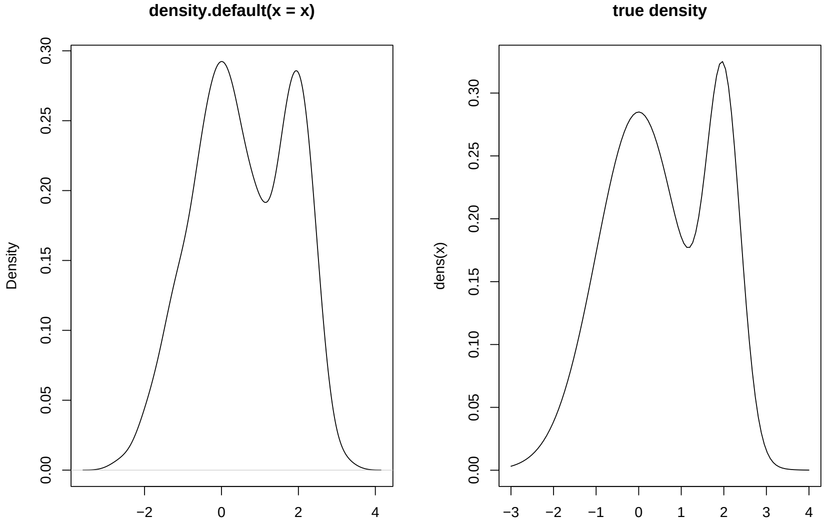
Intro example: Histogram (too many bins)

```
par(mfrow=c(1,2))
hist(x,breaks=500,probability=TRUE)
curve(dens,from=-3,to=4,main="true density")
```



Intro example: KDE (default)

```
par(mfrow=c(1,2))
plot(density(x))
curve(dens,from=-3,to=4,main="true density")
```



Intro example: KDE (default)

- let's look more carefully at what density() returns:

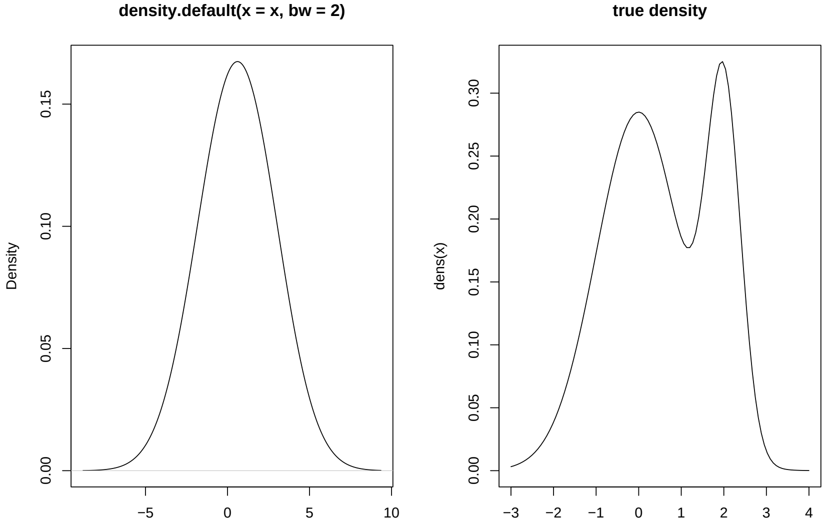
```
density(x)
```

```
##
## Call:
##  density.default(x = x)
##
## Data: x (1400 obs.); Bandwidth 'bw' = 0.2635
##
##        x                 y
##  Min.   :-3.6003   Min.   :1.686e-05
##  1st Qu.:-1.6636   1st Qu.:1.052e-02
##  Median : 0.2732   Median :1.265e-01
##  Mean   : 0.2732   Mean   :1.290e-01
##  3rd Qu.: 2.2099   3rd Qu.:2.323e-01
##  Max.   : 4.1467   Max.   :2.923e-01
```

- Note: "bandwidth"

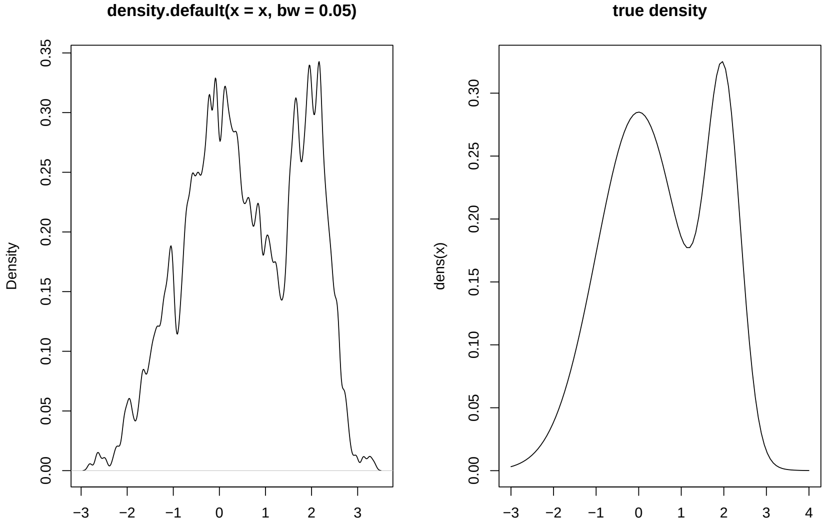
Intro example: KDE (too much smoothing)

```
par(mfrow=c(1,2))
plot(density(x,bw=2.0))
curve(dens,from=-3,to=4,main="true density")
```



Intro example: KDE (too little smoothing)

```
par(mfrow=c(1,2))
plot(density(x,bw=0.05))
curve(dens,from=-3,to=4,main="true density")
```



Objectives here

- Understand the basics of how these two methods work, and thereby their limitations.
- The nature of smoothing (number of bins in hist, bandwidth in KDE)
- Understand the “bias-variance tradeoff”.
- NOT: all the technical details not covered in these notes.

Histograms

Histogram (basics)

- As before; we have a sample $x_1, \dots, x_n$ from some unknown density $f(x)$
- Consider a fixed uniform grid $t_1 < t_2 < \cdots < t_{K+1}$ on the x-axis covering all our observations.
- Uniform: $t_{k+1} - t_k = h$ for each $k = 1, \dots, K$
- Then our estimate of the density $f(x)$ is given by

$$\hat{f}(x) = \frac{\nu_k}{nh} \text{ when } t_k \leq x < t_{k+1}$$

  where ν_k is the number of x_is in the interval $[t_k, t_{k+1})$
- In R: hist() with extra argument: probability=TRUE

Histogram (theoretical basics)

- Note: histogram is a proper density!

$$\int \hat{f}(x)dx = \sum_k \int_{t_k}^{t_{k+1}} \frac{\nu_k}{nh} dx = \sum_k \frac{\nu_k}{nh} h = \frac{1}{n} \underbrace{\sum_k \nu_k}_{=n} = 1$$

- For some fixed x, $\hat{f}(x)$ is an estimator of $f(x)$ that is both *biased* and has *non-zero variance*:
- suppose $t_k \le x < t_{k+1}$, then

$$\nu_k \sim \text{binomial}\left(n, p_k\right),\ p_k = \int_{t_k}^{t_{k+1}} f(y)dy$$

- Recall $E(X) = np,\ Var(X) = np(1-p)$ if $X \sim \text{binomial}(n, p)$

Histogram (bias)

- Bias of $\hat{f}(x)$:

$$E(\hat{f}(x)) = E(\frac{\nu_k}{nh}) = \frac{1}{nh}E(\nu_k) = \frac{1}{nh}np_k = \frac{p_k}{h},$$

  but

$$\frac{p_k}{h} = \underbrace{\frac{1}{h}\int_{t_k}^{t_k+h} f(y)dy}_{\text{mean value of } f(x)} \neq f(x)$$

  unless $f(x)$ is constant! Therefore is $\hat{f}(x)$ a biased estimator of $f(x)$.
- However; choosing the grid so that $x = t_k$, then $\lim_{h\to 0} \frac{1}{h}\int_{t_k}^{t_k+h} f(y)dy = f(t_k) = f(x)$ (if $f(x)$ is smooth), hence we could in principle remove the bias by letting $h \to 0$ (but this would give us infinitely many bins...)
- TAKEAWAY: *bias* grows as h grows (fewer bins, more bias, ref figure above with too few bins).

Histogram (variance)

-Variance of $\hat{f}(x)$:

$$Var(\hat{f}(x)) = Var(\frac{\nu_k}{nh}) = \frac{1}{(nh)^2} Var(\nu_k) = \frac{np_k(1-p_k)}{n^2h^2}$$

$$= \frac{p_k}{nh^2} - \frac{p_k^2}{nh^2} \approx \frac{f(x)}{nh} - \frac{f(x)^2}{n}$$

where the last approximation hold for “small” h (recall $\lim_{h\to 0} \frac{1}{h}\int_{t_k}^{t_k+h} f(x)dx = f(x)$)

- To summarize; for “small” h (and large n), then $Var(\hat{f}(x)) \approx \frac{f(x)}{nh}$.
- TAKEAWAY: Removing bias by letting $h \to 0$ would make the *variance* explode!
- SIDENOTE: by choosing h sufficiently large, we get a single bin with height that has zero variance.

bias-variance tradeoff

- Two takeaways: we should choose *h small enough so that the bias is not too large*, but we should choose *h large enough so that the variance is not too large*...
- One approach is based on minimizing the mean-squared error (MSE) (for fixed x):

$$MSE(\hat{f}(x)) = E(\hat{f}(x) - f(x))^2 = Var(\hat{f}(x)) + \underbrace{[E(\hat{f}(x) - f(x))]^2}_{= \text{squared bias}}$$

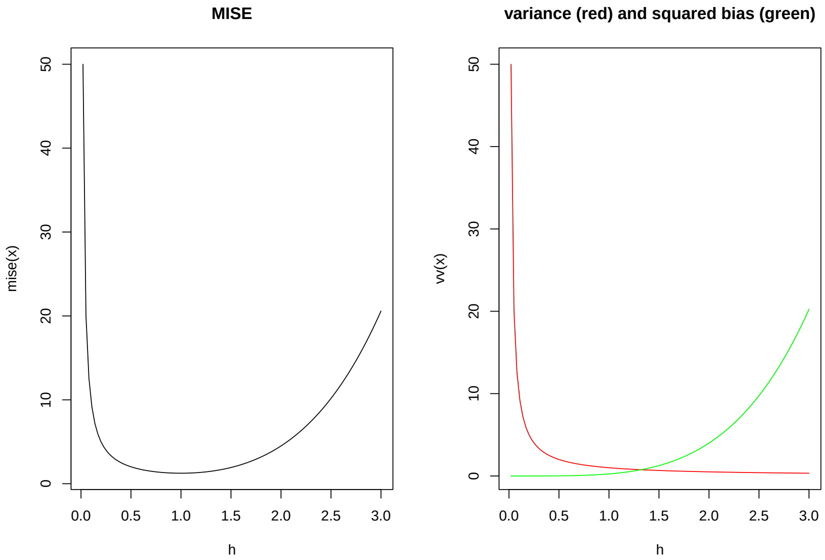
or more generally mean-integrated squared error (MISE):

$$MISE(\hat{f}) = \int MSE(\hat{f}(x))dx$$

- Always a tradeoff between variance term and squared bias term.

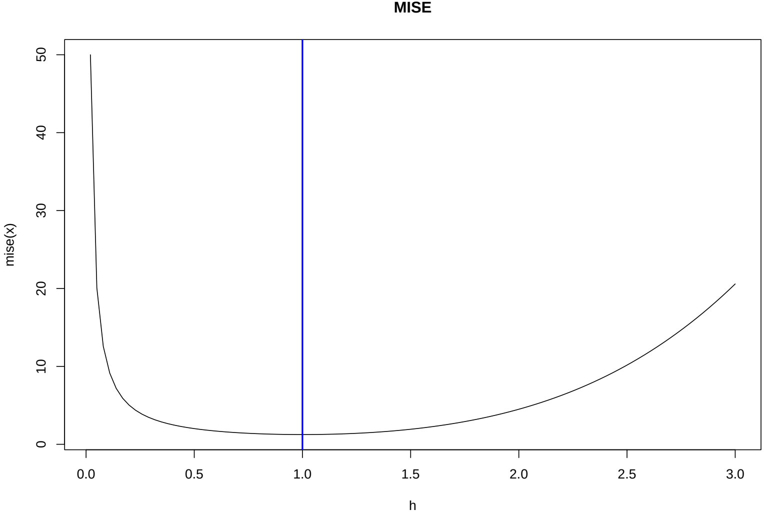
bias-variance tradeoff

- MSE or MISE as function of h typically looks something like this:



choose a tradeoff

- We choose a value of h where the MISE is minimized:



Rules for choosing h

- Many rules exist for choosing h, depending on assumptions made.
- For the histogram, it can be shown that (for small h and large n)
$$MISE = \frac{1}{nh} + \frac{h^2}{12}\int f'(x)^2 dx + O\left(\frac{1}{n} + h^3\right)$$
- Minimizing the leading terms of this expression as a function of h yields
$$h_n^* = \left(\frac{6}{n\int f'(x)^2 dx}\right)^{1/3}$$
- So far not operational as it depends on unknown quantiy $\int f'(x)^2 dx$ - need to make further assumptions

Rules for choosing h - normal assumption

- Assuming data $x_1, \ldots, x_n$ are normal, then

$$\int f'(x)^2 dx = \int_{-\infty}^{\infty} \left[\frac{d}{dx} \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-(x-\mu)^2/(2\sigma^2)) \right]^2 dx$$

$$\approx 0.1410473959\sigma^{-3}$$

- Yields Scott's Normal Reference rule:

$$\hat{h} = (6/0.1410473959)^{1/3}(\sigma^3)^{1/3} n^{-1/3} \approx 3.49\hat{\sigma} n^{-1/3}$$

where $\hat{\sigma}$ is an estimator of the standard deviation of the data (typically s).

Sturges' Rule

- Very common.
- Based on more heuristic reasoning; yields

$$\hat{h} = \frac{\max_i x_i - \min_i x_i}{1 + \log_2(n)}$$

where $\log_2$ is the base 2 logarithm (log2() in R).

example - Scott's Normal Reference rule:

- Note: R rounds h to a "nice" number...

```
h.out <- hist(x,breaks="Scott",plot=FALSE)
print(paste0("h used by R : ",
             h.out$breaks[2]-h.out$breaks[1]))
```

```
## [1] "h used by R : 0.5"
```

```
print(paste0("h suggested by Scott's rule : ",
             3.49*sd(x)*length(x)^(-1/3) ))
```

```
## [1] "h suggested by Scott's rule : 0.388943633178186"
```

example - Sturges' Rule:

- Note: R rounds h to a "nice" number...

```
h.out <- hist(x,plot=FALSE) #default method in R
print(paste0("h used by R : ",
             h.out$breaks[2]-h.out$breaks[1]))
```

```
## [1] "h used by R : 0.5"
```

```
print(paste0("h suggested by Sturges' Rule : ",
             (max(x)-min(x))/(1+log2(length(x)))))
```

```
## [1] "h suggested by Sturges' Rule : 0.538451605446172"
```

Kernel density estimation (KDE)

Basics of KDE

- Let $K(\cdot)$ be some symmetric probability density function with mean=0, variance=1, e.g. standard normal pdf.
- I.e. $\int K(x)dx = 1$, $\int xK(x)dx = 0$, $\int x^2K(x)dx = 1$.
- The KDE for some independent sample $X_1, \ldots, X_n$ is then given by
  $$\hat{f}_K(x) = \frac{1}{n}\sum_{i=1}^{n}\frac{1}{h}K\left(\frac{x - X_i}{h}\right)$$
  $h > 0$ is the "bandwidth" (smoothing parameter).
- Much more flexible than histograms, but more things can also go wrong... (both have their pros and cons)
- Note: $\int \hat{f}_K(x)dx = 1$ i.e. proper density.

Example

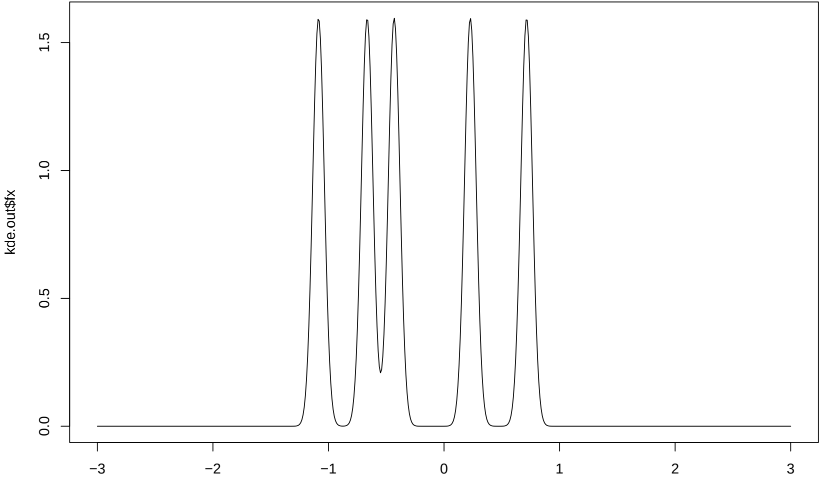
- "Lazy" implementation based on Gaussian kernel:

```
my.kde <- function(x,X,h){ # allow for a vector of x-s
  fx <- numeric(length(x)) # allocate space
  for(i in 1:length(x)){
    #eqn in previous slide
    fx[i] <- mean(dnorm(x[i],mean=X,sd=h))
  }
  return(list(x=x,fx=fx,h=h))
}
```

- Note: if K is the $N(0,1)$-pdf, then $h^{-1}K(h^{-1}(x - X_i))$ is the $N(X_i, h^2)$-pdf evaluated at x.

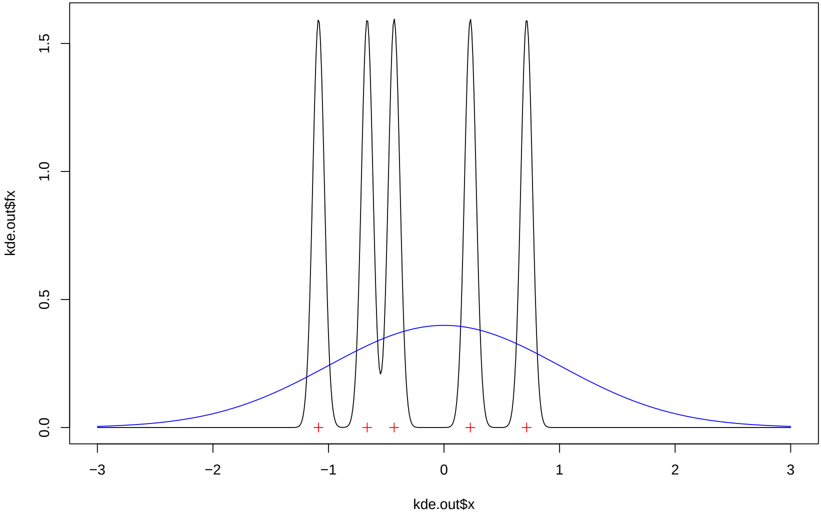
Example continued

```
X <- rnorm(5) # tiny sample for illustration
x.grid <- seq(from=-3.0,to=3.0,by=0.01)
kde.out <- my.kde(x=x.grid,X=X,h=0.05)
plot(kde.out$x,kde.out$fx,type="l")
```



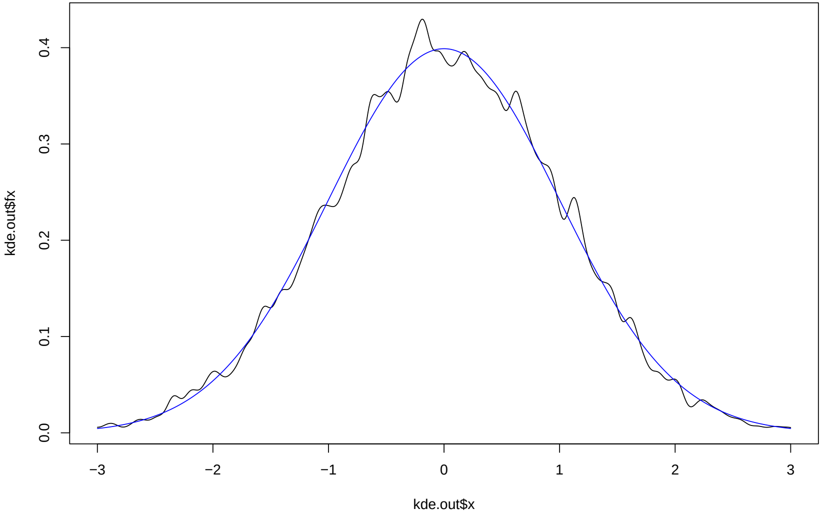
Example continued

```
plot(kde.out$x,kde.out$fx,type="l")
points(X,0*X,pch=3,col="red")
lines(x.grid,dnorm(x.grid),col="blue")
```



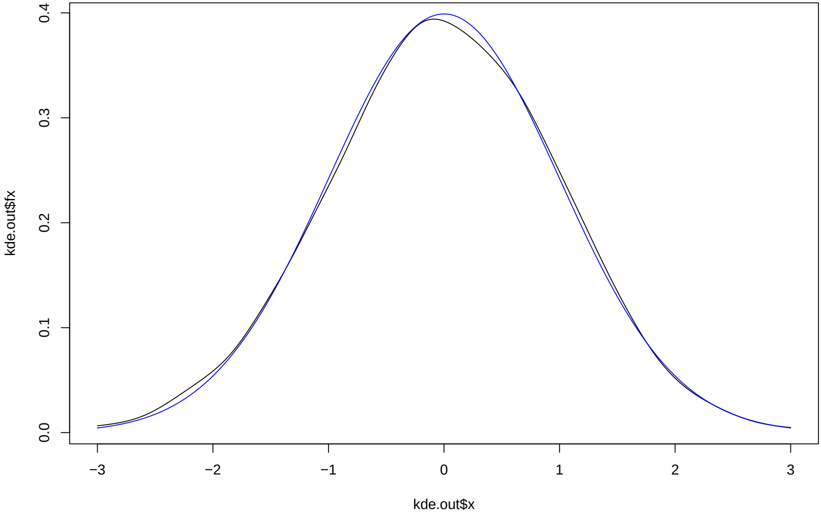
Example continued, More data

```
X <- rnorm(5000); kde.out <- my.kde(x=x.grid,X=X,h=0.05)
plot(kde.out$x,kde.out$fx,type="l")
lines(x.grid,dnorm(x.grid),col="blue")
```



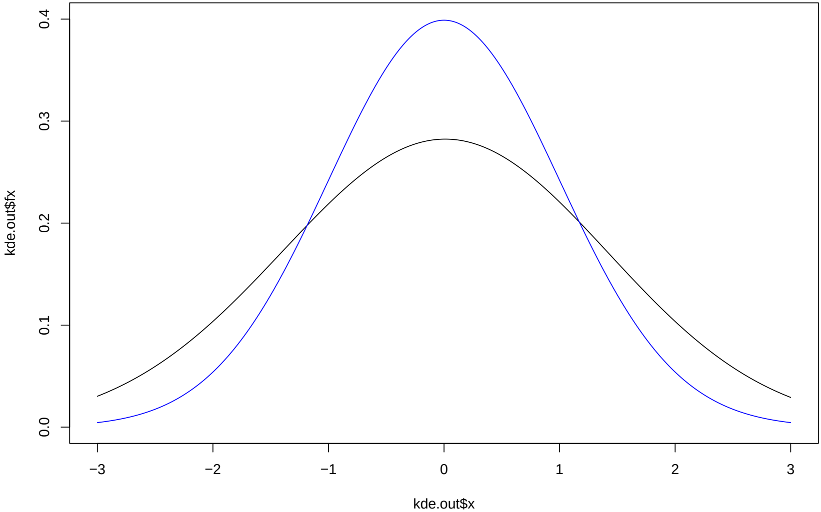
Example continued, More smoothing

```
kde.out <- my.kde(x=x.grid,X=X,h=0.2)
plot(kde.out$x,kde.out$fx,type="l")
lines(x.grid,dnorm(x.grid),col="blue")
```



Example continued, Too much smoothing

```
kde.out <- my.kde(x=x.grid,X=X,h=1)
plot(kde.out$x,kde.out$fx,type="l",ylim=c(0,0.4))
lines(x.grid,dnorm(x.grid),col="blue")
```



Choosing the bandwidth

- The bandwidth parameter h plays very much the same role as the bin-width in the histogram.
- Generally:
  - Small h leads to low bias but high variance.
  - Large h leads to low variance and high bias.
- Note:

$$E(\hat{f}_K(x)) = \int h^{-1}K(h^{-1}(x-y))f(y)dy = f(x) + \underbrace{\frac{h^2 f''(x)}{2} + O(h^4)}_{bias}$$

for small $h > 0$. For small h and large n, $Var(\hat{f}_K(x)) \propto (nh)^{-1}$.

MISE for the KDE:

- Still assuming $\int x^2 K(x)dx = 1$, then the (AMISE) optimal bandwidth is

$$\hat{h} = \left[\frac{\int (K(x))^2 dx}{\int (f''(x))^2 dx} \right]^{1/5} n^{-1/5}$$

  Again not directly applicable since it depends on the unknown $f''(x)$.

- For standard normal K and $f(x)$ corresponding to a $N(\mu, \sigma^2)$-distribution;

$$\int (K(x))^2 dx = \frac{1}{2\sqrt{\pi}}, \int (f''(x))^2 dx \approx 0.2115710938\sigma^{-5}$$

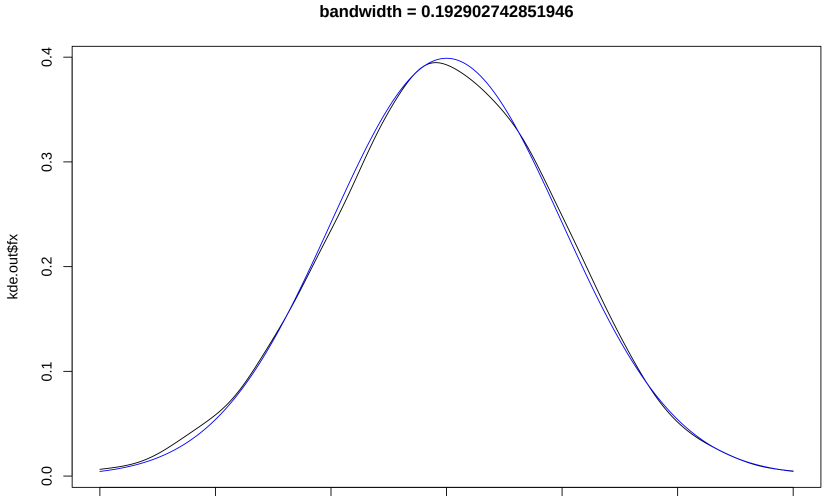
  we get "the rule of thumb": $\hat{h} \approx 1.06\hat{\sigma} n^{-1/5}$ where $\hat{\sigma}$ is an estimate of σ.

Back to example; function redefinition

```
my.kde <- function(x,X,h=1.06*sd(X)*length(X)^(-0.2)){
  fx <- numeric(length(x)) # allocate space
  for(i in 1:length(x)){
    fx[i] <- mean(dnorm(x[i],mean=X,sd=h))
  }
  return(list(x=x,fx=fx,h=h))
}
```

Try new function

```
kde.out <- my.kde(x=x.grid,X=X)
plot(kde.out$x,kde.out$fx,type="l",
     main=paste0("bandwidth = ",kde.out$h))
lines(x.grid,dnorm(x.grid),col="blue")
```

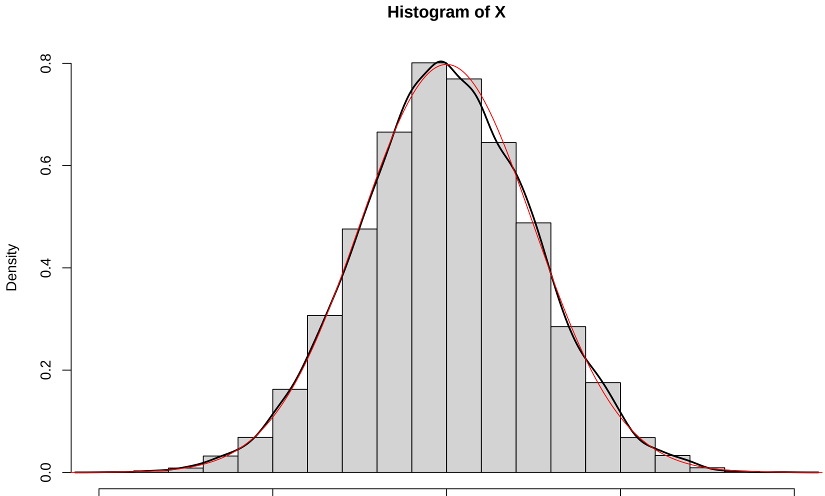


KDE in practice

- In practice, one typically uses the built in density()-function for KDEs.
- density() allows for different kernels K and has automatic bandwidth selection.
- BUT RECALL: **KDEs are (generally biased) estimators of the density $f(x)$, not the density itself**.
- Bad things can happen if the data deviates much from normality (which the histgram is more robust against).

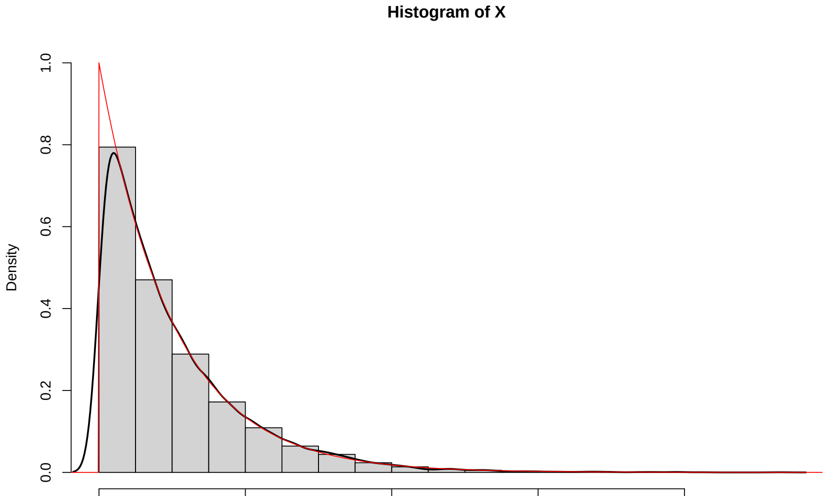
Examples: Normal

```
X <- rnorm(10000,mean=2,sd=0.5)
hist(X,probability=TRUE);lines(density(X),lwd=2)
xg <- seq(from=-1,to=5,by=0.01)
lines(xg,dnorm(xg,mean=2,sd=0.5),col="red")
```



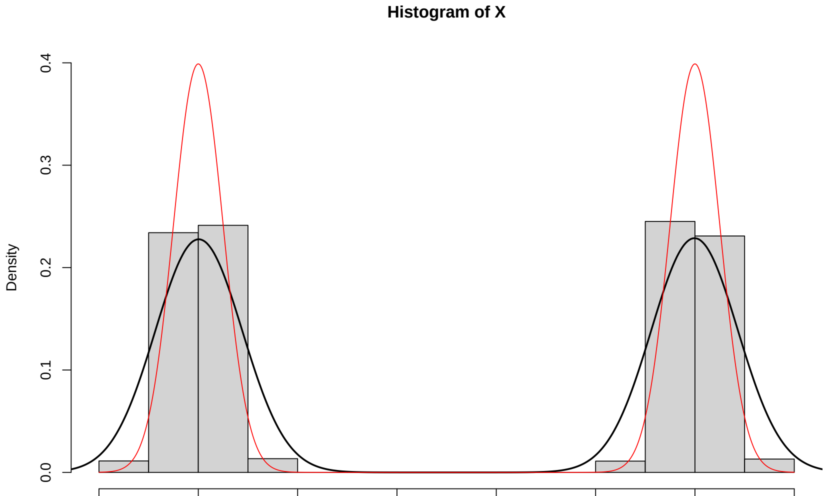
Examples: Exponential (KDE fails around $x = 0$)

```
X <- rexp(10000)
hist(X,probability=TRUE,ylim=c(0,1));lines(density(X),lwd=2
xg <- seq(from=-1,to=15,by=0.01)
lines(xg,dexp(xg),col="red")
```



Examples: Normal mixture (both fail really. . .)

```
X <- c(rnorm(5000,sd=0.5),rnorm(5000,mean=10,sd=0.5))
hist(X,probability=TRUE,ylim=c(0,0.4));lines(density(X),lwd
xg <- seq(from=-2,to=12,by=0.01)
lines(xg,0.5*dnorm(xg,sd=0.5)+0.5*dnorm(xg,mean=10,sd=0.5),
```



Last takeaways;

- Be careful with- and critical to density estimation.
- If additional information is available, this may inform your choice of algorithm.
- E.g. if it known that $X > 0$, then use boundary kernels (page 359).
- Large literature on e.g. different choices of K, higher dimensions, and so on.